FINIRAG

Document Intelligence Platform

AI Engineer Internship • Technical Case Study

PDF / JPG / PNG

4 document types

AI + rules



Candidate

Poli Reddy

Problem Statement

Turn heterogeneous financial documents into trustworthy, reviewable structured data.

Layout-heavy inp

Scanned images, vector-outlined PDFs, tables and multi-page statements.

Semantic ambigui

Missing ≠ zero ≠ unreadable; tax-inclusive totals, discounts and units need context.

Financial correc

Extraction alone is insufficient: arithmetic must be checked deterministically.

Operational deli

Persist attempts, expose APIs, provide history/dashboard and deploy the system.

Input: PDF/JPG/PNG • User selects type: Invoice / Balance Sheet / P&L / Cash Flow • Max 3 pages • Max 10 MB

Solution Overview

A provider-abstracted pipeline separates AI extraction from deterministic financial truth.

01

File Gate

Signature, readability, size + page checks

02

Text / OCR

Native PDF probe or Mistral OCR

03

Structured AI

Gemini primary Groq fallback

04

Canonical Model

Pydantic + normalization + evidence

05

Financial Rules

Decimal arithmetic PASS / FAIL / N/A

06

Persist + Serve

PostgreSQL + FastAPI + dashboard

AI reads

LLMs handle visual/text understanding and schema extractio

Rules decide

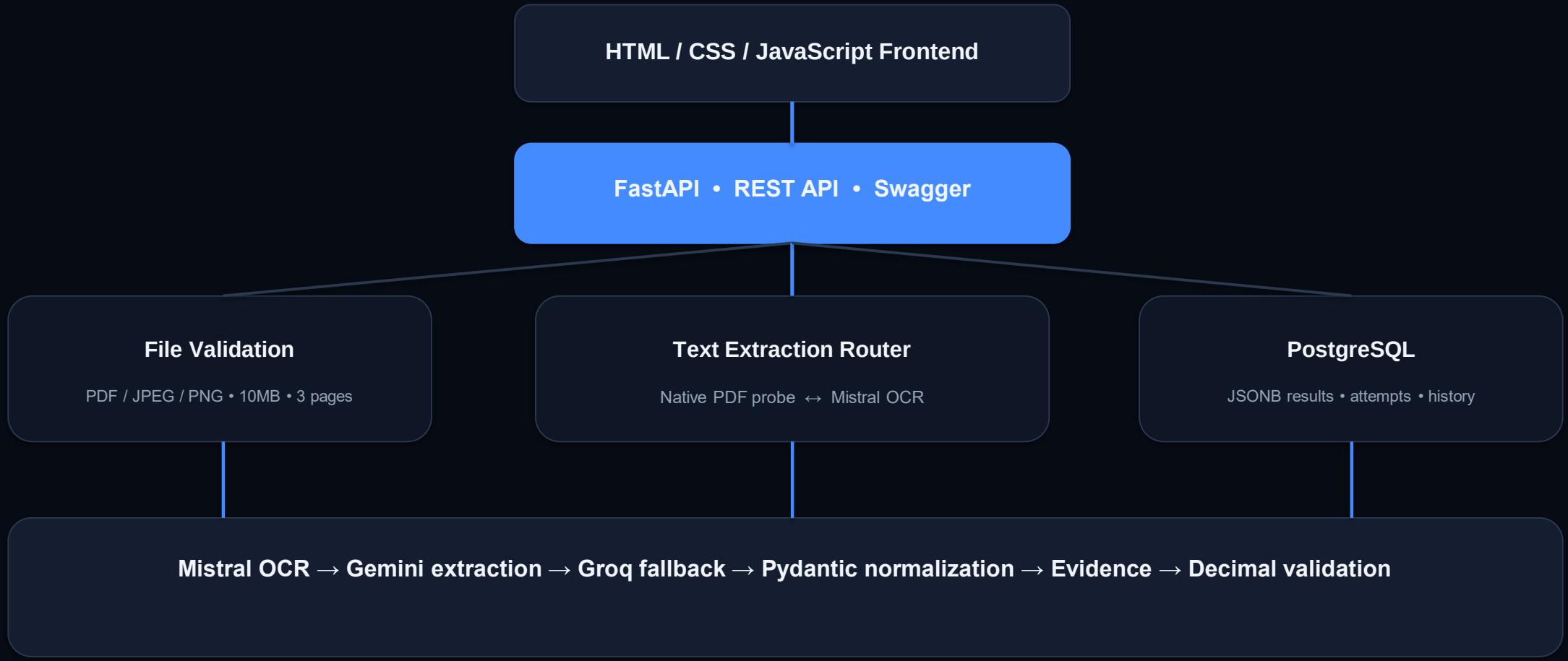
Python Decimal validators own arithmetic and reconciliation.

Evidence stays

Original representation, page and source text remain attached.

Architecture Diagram

Single FastAPI service serves the dashboard, API and processing pipeline; PostgreSQL stores attempts.



Document Processing Pipeline

Each stage has a clear responsibility; failures remain explicit instead of being silently converted.

01 — Upload
Read bytes
sanitize filename

02 — Validate
Magic/signature
readability • limits

03 — Extract text
Native PDF
or Mistral OCR

04 — AI extract
Typed JSON
all meaningful fields

05 — Normalize
Decimal values
units • signs • status

06 — Ground
Source text
page + evidence

07 — Validate
Financial formulas
PASS / FAIL / N/A

08 — Persist
PostgreSQL
latest + history

Field state model

- EXPLICIT
- EXPLICIT_ZERO
- MISSING
- UNREADABLE

Evidence payload

Original source text
Page number • optional block/box

Persistence

One row per processing attempt
Latest-by-name retrieval

AI / OCR Extraction Approach

Mistral OCR handles visual document reading; Gemini structures the result; Groq provides fallback capacity.

Primary OCR

Mistral OCR

49/50 dataset docs

Layout-aware OCR blocks • page text • block confidence • bound

Primary LLM

Gemini

47/50 dataset docs

Structured extraction into document-specific Pydantic schemas

Fallback LLM

Groq

3/50 dataset docs

Provider fallback for quota/rate-limit/parse-oriented failures

Routing observed in the 50-document run

49 Mistral OCR

1 native PDF text

Trust boundary

LLM = extraction authority for structure. Python Decimal = arithmetic authority.

Financial Validation

Deterministic Decimal checks distinguish genuine inconsistencies from extraction ambiguity.

Invoice / Receipt

- quantity × unit price
- discount-aware line totals
- subtotal / tax / total
- tax-inclusive readings
- cash − total = change

Balance Sheet

- assets ≈ capital + liabilities
- asset component sums
- liability component sums
- each period independently

Profit & Loss

- income components
- expenditure components
- profit / minority reconciliation
- appropriation equation
- carried-forward balance

Cash Flow

- operating / investing / financing
- section component sums
- opening + net increase
- FX / amalgamation guards
- closing cash reconciliation

Dataset regression example **batch3-1495** **$3,480 line sum** **vs $5,257 total** **FAIL**

The validator preserves the inconsistency instead of weakening rules to force PASS.

API + Database + Deployment

A single Render web service serves the frontend and API, backed by Render PostgreSQL.

REST API

POST	/api/v1/documents/process	Upload + process
GET	/api/v1/documents/{document_name}	Latest result
GET	/api/v1/documents	History / attempts
GET	/api/v1/health	API + DB health

PostgreSQL

documents
JSONB payloads

Every processing attempt is retained

latest-by-name

Render

Web Service • free plan

PostgreSQL • free plan

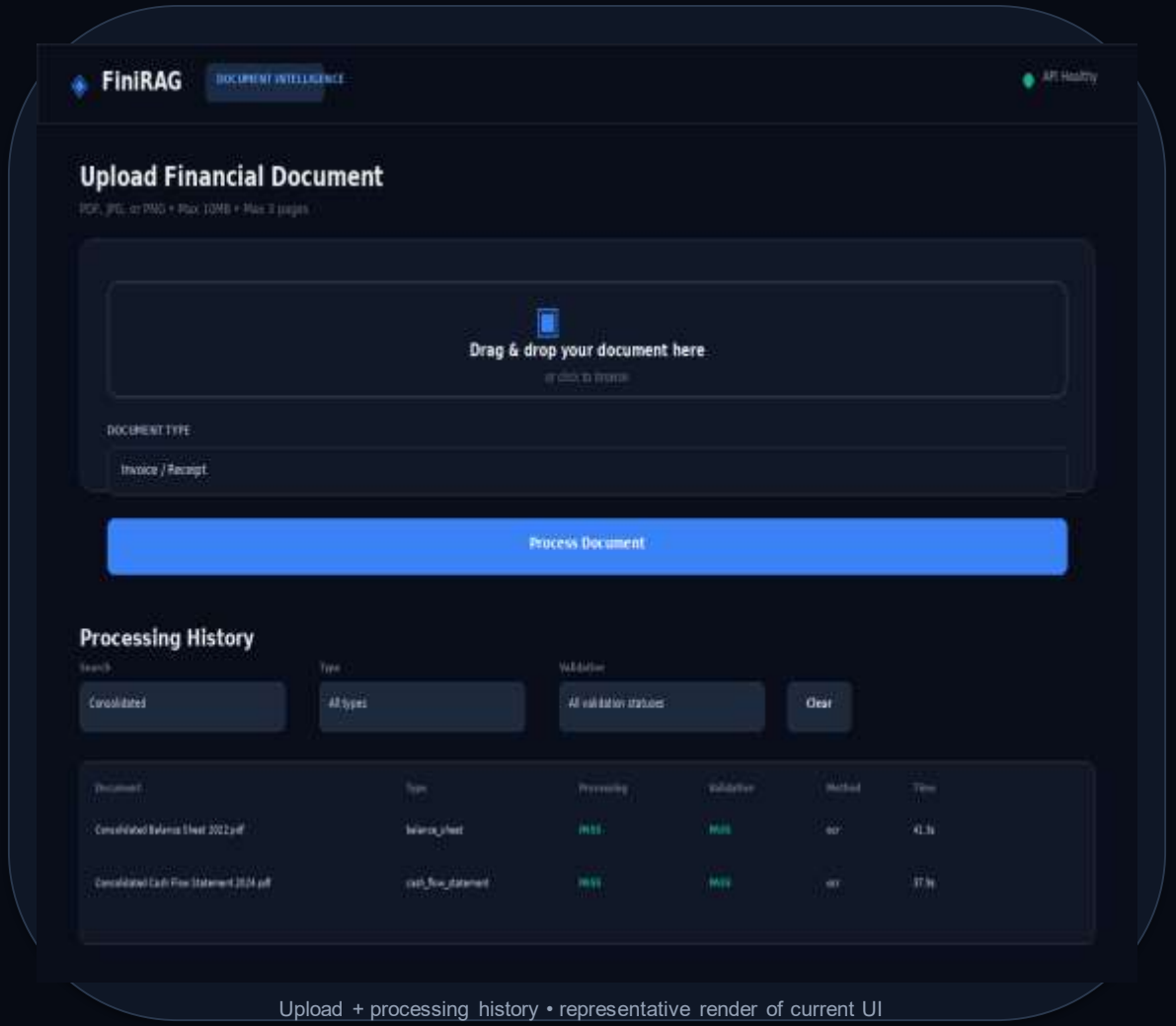
Health check
/api/v1/health

Swagger: /docs
Frontend: /

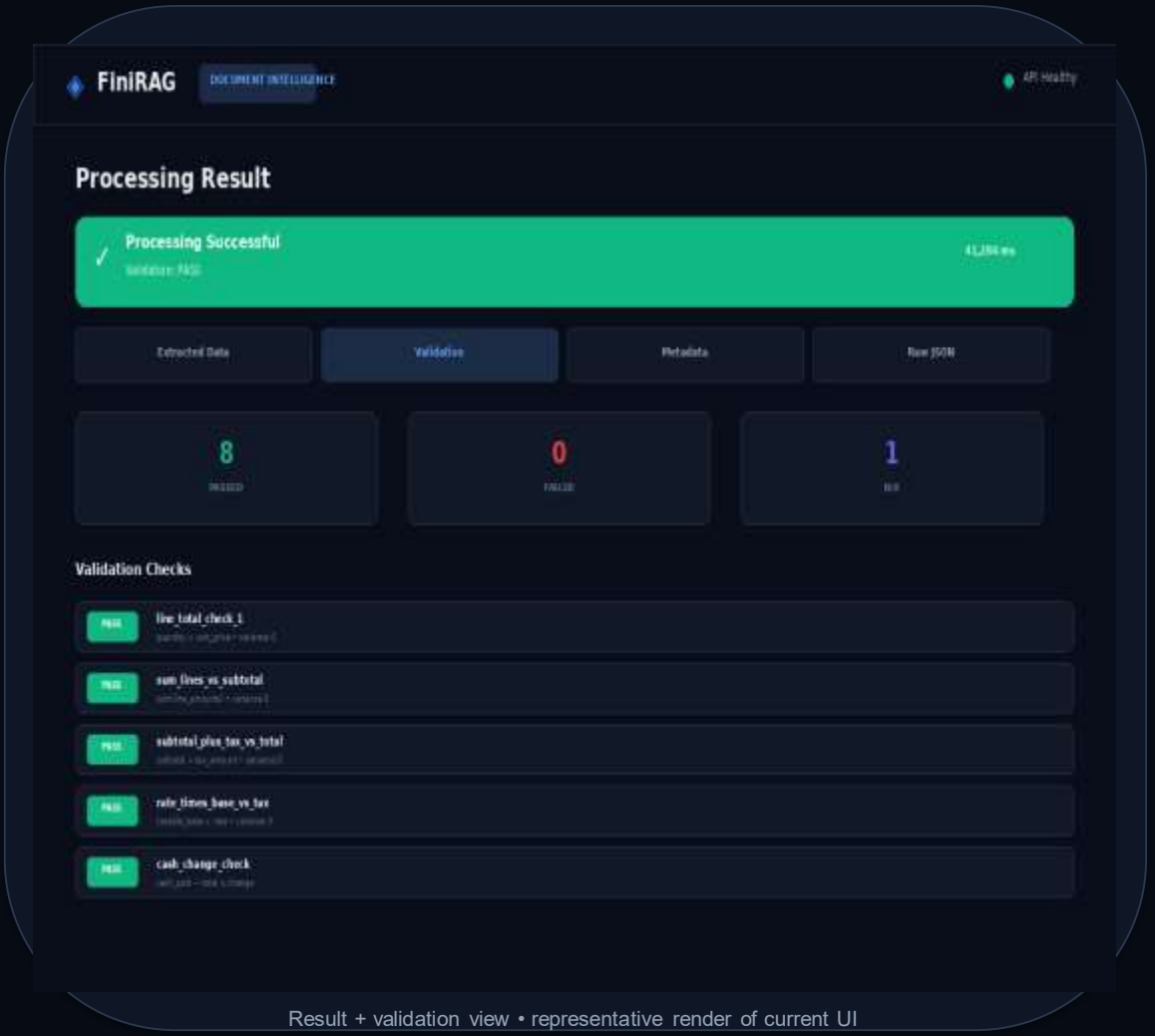
Operational boundary: secrets via environment variables • no raw files persisted • historical attempts preserved

Frontend Screenshots / Demo

Implemented HTML/CSS/JavaScript dashboard: upload, processing result, validation, metadata, raw JSON and history.



Upload + processing history • representative render of current UI



Result + validation view • representative render of current UI

Testing + Limitations + Future Improvements

The current repository is feature-complete for the case study, with clear production hardening opportunities.

TESTING & EVALUATION

50/50
processing PASS

47/50
validation PASS

290
test functions

3
intended FAILs

350 PASS • 10 FAIL • 56 N/A

2,665 field slots measured
980 line items extracted

CURRENT LIMITATIONS

- Evidence has source text + page, but block-level verification is not yet reliable for every field.
- Provider latency and free-tier availability can vary.
- History search/filtering is client-side over the loaded dashboard page.
- Authentication, rate limiting and background jobs are deferred.
- Database migrations are deferred in favor of startup table creation for the prototype.

FUTURE / PRODUCTION

- Verified OCR block-to-field grounding + bounding boxes.
- Background queue/workers for long-running document processing.
- Alembic migrations + stronger DB operations.
- Authentication, authorization, rate limits and audit logging.
- Provider circuit breakers, observability and staging E2E tests.

Bottom line: correct extraction + explicit uncertainty + deterministic validation — without weakening rules to manufacture PASS results.